final_LinearSystemMismatchTestDiagnostics

March 1, 2026

[1]:
```
import sys
import os

notebook_path = os.getcwd()
parent_dir = os.path.dirname(notebook_path)
project_root = os.path.dirname(parent_dir)
if project_root not in sys.path:
    sys.path.insert(0, project_root)
```

[2]:
```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
import matplotlib.pyplot as plt
from copy import deepcopy
```

[3]:
```
import state_NN_models
import Filters
import utils
import Systems
from utils import losses, trainer, utils
from torch.utils.data import TensorDataset, DataLoader, random_split
from state_NN_models.StateBayesianKalmanNet import StateBayesianKalmanNet
from state_NN_models.StateKalmanNet import StateKalmanNet
from state_NN_models.StateKalmanNetWithKnownR import StateKalmanNetWithKnownR
```

[4]:
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Používané zařízení: {device}")
```

```
Používané zařízení: cuda
```

[5]:
```
import math
import torch

state_dim_2d = 2
obs_dim_2d = 2

# ==============================================================
# 1. SKUTEČNÝ SYSTÉM (2D Pozice - Random Walk)
```

```python
# ==============================================================
# Matice identity. Vlastní čísla jsou přesně 1.0.
# Stav nereprezentuje rychlost, jen X a Y pozici, která se posouvá pomocí šumu␣
↪Q.
F_true_2d = torch.tensor([[1.0, 0.0],
                          [0.0, 1.0]])
print("F_true_2d (Skutečná dynamika):\n", F_true_2d)

H_true_2d = torch.eye(obs_dim_2d)

Q_true_2d = torch.eye(state_dim_2d) * 0.1 # Šum procesu pohání trajektorii
R_true_2d = torch.eye(obs_dim_2d) * 2.0   # Hodně zašuměná měření

Ex0_true_2d = torch.tensor([[1.0], [0.0]])
P0_true_2d = torch.eye(state_dim_2d) * 1.5

# ==============================================================
# 2. MODEL SYSTÉMU (Zavedení čisté rotace - Mismatch)
# ==============================================================
delta_alpha_deg = 10.0
delta_alpha_rad = delta_alpha_deg * math.pi / 180.0

# Vytvoření 2D rotační matice R(Δ )
rotation_matrix = torch.tensor([
    [math.cos(delta_alpha_rad), -math.sin(delta_alpha_rad)],
    [math.sin(delta_alpha_rad),  math.cos(delta_alpha_rad)]
])

# Model předpokládá, že se poloha v každém kroku pootočí o 10 stupňů
# Vlastní čísla čisté rotační matice mají velikost přesně 1.0. EXPLOSE SE␣
↪NEKONÁ!
F_model_2d = torch.matmul(rotation_matrix, F_true_2d)
print("\nF_model_2d (S chybou rotace o 10°):\n", F_model_2d)

H_model_2d = H_true_2d
Q_model_2d = Q_true_2d
R_model_2d = R_true_2d

Ex0_model_2d = Ex0_true_2d
P0_model_2d = P0_true_2d

# Incializace sys_true a sys_model zůstává stejná...
# ==============================================================
# 3. INICIALIZACE OBJEKTŮ SYSTÉMU
# ==============================================================
print("\nInicializuji 2D Linear_Canonical systém (Realita vs. Model)...")
sys_true = Systems.DynamicSystem(
```

```
    state_dim=state_dim_2d, obs_dim=obs_dim_2d,
    Ex0=Ex0_true_2d, P0=P0_true_2d,
    Q=Q_true_2d, R=R_true_2d,
    F=F_true_2d, H=H_true_2d,
    device=device
)

sys_model = Systems.DynamicSystem(
    state_dim=state_dim_2d, obs_dim=obs_dim_2d,
    Ex0=Ex0_model_2d, P0=P0_model_2d,
    Q=Q_model_2d, R=R_model_2d,
    F=F_model_2d, H=H_model_2d,
    device=device
)

print("... 2D systémy úspěšně inicializovány s modelovým mismatchem.")
```

```
F_true_2d (Skutečná dynamika):
 tensor([[1., 0.],
        [0., 1.]])

F_model_2d (S chybou rotace o 10°):
 tensor([[ 0.9848, -0.1736],
        [ 0.1736,  0.9848]])

Inicializuji 2D Linear_Canonical systém (Realita vs. Model)…
… 2D systémy úspěšně inicializovány s modelovým mismatchem.
```

[6]:
```
TRAIN_SEQ_LEN = 50      # Krátké sekvence pro stabilní trénink (TBPTT)
VALID_SEQ_LEN = 200      # Stejná délka pro konzistentní validaci
TEST_SEQ_LEN = 1000      # Dlouhé sekvence pro testování generalizace

NUM_TRAIN_TRAJ = 500   # Hodně trénovacích příkladů
NUM_VALID_TRAJ = 50    # Dostatek pro spolehlivou validaci
NUM_TEST_TRAJ = 100     # Pro robustní vyhodnocení

BATCH_SIZE = 32         # Dobrý kompromis

x_train, y_train = utils.generate_data(sys_true,␣
 ↪num_trajectories=NUM_TRAIN_TRAJ, seq_len=TRAIN_SEQ_LEN)
x_val, y_val = utils.generate_data(sys_true, num_trajectories=NUM_VALID_TRAJ,␣
 ↪seq_len=VALID_SEQ_LEN)
x_test, y_test = utils.generate_data(sys_true, num_trajectories=NUM_TEST_TRAJ,␣
 ↪seq_len=TEST_SEQ_LEN)

train_dataset = TensorDataset(x_train, y_train)
val_dataset = TensorDataset(x_val, y_val)
```

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
```

[7]:
```
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
import numpy as np
import os
import random
import csv
from datetime import datetime
import pandas as pd
from copy import deepcopy

model_config = {
    "hidden_size_multiplier": 10,
    "output_layer_multiplier": 4,
    "num_gru_layers": 1,
    "init_min_dropout": 0.25,#0.3
    "init_max_dropout": 0.35 #0.4
}

train_config = {
    "total_train_iter": 1200,
    "learning_rate": 1e-4,
    "clip_grad": 10.0,
    "J_samples": 10,
    "validation_period": 20,
    "logging_period": 5,
    "warmup_iterations": 100, # Trénuj prvních 100 iterací jen na MSE
    "store_model_based_on_hybrid_score": True,
    "calibration_parameter": 0.5
}

#␣
↪==============================================================================
# KROK 3: SPUŠTĚNÍ JEDNOHO TRÉNINKOVÉHO BĚHU NEBO NAČTENÍ HOTOVÉHO MODELU
#␣
↪==============================================================================
print("device:", device)
print("="*80)
print("Připravuji Bayesian KalmanNet (BKN)...")
print(f"Parametry modelu: {model_config}")
print(f"Parametry tréninku: {train_config}")
print("="*80)
```

```python
# Nastavení seedu pro reprodukovatelnost tohoto běhu
torch.manual_seed(42)
np.random.seed(42)
random.seed(42)

# Definice cesty ke složce a souboru
save_dir = "LinearSystemWeightsMismatch"
os.makedirs(save_dir, exist_ok=True)
weights_path = os.path.join(save_dir, "best_bkn_model_weights_mismatch.pth")

# Vytvoření modelu (potřebujeme ho inicializovat v obou případech)
state_bkn_knet = StateBayesianKalmanNet(
    sys_model,
    device=device,
    weight_init=False,
    **model_config
).to(device)

# Podmínka pro načtení nebo trénink
if os.path.exists(weights_path):
    print(f"\nINFO: Uložené váhy nalezeny v '{weights_path}'.")
    print("Přeskakuji trénink a načítám hotový model...")

    # Načtení vah
    state_bkn_knet.load_state_dict(torch.load(weights_path,␣
 ↪map_location=device))
    trained_model = state_bkn_knet

else:
    print("\nINFO: Váhy nebyly nalezeny. Spouštím plnohodnotný tréninkový běh...
 ↪")

    # Spuštění tréninku
    results = trainer.
 ↪training_session_trajectory_with_gaussian_nll_training_fcn(
        model=state_bkn_knet,
        train_loader=train_loader,
        val_loader=val_loader,
        device=device,
        **train_config
    )

    # `run_training_session` automaticky načte nejlepší model zpět,
    # takže `state_bkn_knet` nyní obsahuje váhy nejlepšího modelu.
    trained_model = results['final_model']

    # Uložení vah pro příští použití
```

```
    torch.save(trained_model.state_dict(), weights_path)
    print(f"\nINFO: Váhy modelu byly úspěšně uloženy do: {weights_path}")

    # Výpis metrik pouze pokud jsme trénovali
    print("\n" + "="*80)
    print("TRÉNINK DOKONČEN - FINÁLNÍ VÝSLEDKY Z NEJLEPŠÍHO MODELU")
    print("="*80)
    print(f"Nejlepší model byl nalezen v iteraci: {results['best_iter']}")
    print(f"Nejlepší dosažený validační ANEES: {results['best_val_anees']:.4f}")
    print("--- Metriky odpovídající tomuto nejlepšímu modelu ---")
    print(f"  MSE na validační sadě:       {results['best_val_mse']:.4f}")
    print(f"  NLL na validační sadě:       {results['best_val_nll']:.4f}")
    print("="*80)

# Prepnutí do evaluačního módu na konci bloku (klíčové pro BKN kvůli Dropoutu)
trained_model.eval()
print("\nModel BKN je plně připraven v proměnné 'trained_model'.")
```

```
device: cuda
================================================================================
Připravuji Bayesian KalmanNet (BKN)…
Parametry modelu: {'hidden_size_multiplier': 10, 'output_layer_multiplier': 4,
'num_gru_layers': 1, 'init_min_dropout': 0.25, 'init_max_dropout': 0.35}
Parametry tréninku: {'total_train_iter': 1200, 'learning_rate': 0.0001,
'clip_grad': 10.0, 'J_samples': 10, 'validation_period': 20, 'logging_period':
5, 'warmup_iterations': 100, 'store_model_based_on_hybrid_score': True,
'calibration_parameter': 0.5}
================================================================================
loaded with input normalization

INFO: Uložené váhy nalezeny v
'LinearSystemWeightsMismatch/best_bkn_model_weights_mismatch.pth'.
Přeskakuji trénink a načítám hotový model…

Model BKN je plně připraven v proměnné 'trained_model'.
```

[8]:
```
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
import numpy as np
import os
import random
import csv
from datetime import datetime
import pandas as pd
from copy import deepcopy
# Nastavení seedu pro reprodukovatelnost tohoto běhu
```

```python
torch.manual_seed(42)
np.random.seed(42)
random.seed(42)

save_dir = "LinearSystemWeightsMismatch"
weights_path = os.path.join(save_dir, "best_knet_model_weights_mismatch.pth")

# Vytvoření cílové složky (pokud ještě neexistuje, nic se nestane)
os.makedirs(save_dir, exist_ok=True)


state_knetR = StateKalmanNet(sys_model, device=device,␣
 ↪hidden_size_multiplier=12,returns_covariance=True).to(device)

if os.path.exists(weights_path):
    print(f"INFO: Uložené váhy nalezeny v '{weights_path}'.")
    print("Přeskakuji trénink a načítám hotový model...")

    # Načtení vah (map_location zajistí kompatibilitu, pokud bys např. trénoval␣
 ↪na GPU a načítal na CPU)
    state_knetR.load_state_dict(torch.load(weights_path, map_location=device))

else:
    print("INFO: Váhy nebyly nalezeny. Spouštím trénink modelu...")

    # Spuštění tréninku
    trainer.train_state_KalmanNet(
        model=state_knetR,
        train_loader=train_loader,
        val_loader=val_loader,
        device=device,
        epochs=200,
        lr=1e-4,
        early_stopping_patience=30,
        weight_decay=1e-3
    )

    # Po úspěšném tréninku váhy rovnou uložíme pro příští spuštění
    torch.save(state_knetR.state_dict(), weights_path)
    print(f"Trénink dokončen. Váhy byly bezpečně uloženy do '{weights_path}'.")

# Prepnutí do evaluačního módu (dobrá praxe před testováním)
state_knetR.eval()
```

```
DEBUG: Layer 'output_final_linear.0' initialized near zero (Start K=0).
INFO: Uložené váhy nalezeny v
'LinearSystemWeightsMismatch/best_knet_model_weights_mismatch.pth'.
```

```
Přeskakuji trénink a načítám hotový model…
```

[8]:
```
StateKalmanNet(
  (dnn): DNN_KalmanNet(
    (input_layer): Sequential(
      (0): Linear(in_features=8, out_features=384, bias=True)
      (1): ReLU()
    )
    (gru): GRU(384, 8)
    (output_hidden_layer): Sequential(
      (0): Linear(in_features=8, out_features=16, bias=True)
      (1): ReLU()
    )
    (output_final_linear): Sequential(
      (0): Linear(in_features=16, out_features=4, bias=True)
    )
  )
)
```

[9]:
```python
import torch
import torch.nn.functional as F
import numpy as np
from torch.utils.data import TensorDataset, DataLoader
import Filters
from copy import deepcopy  # <-- ZÁSADNÍ IMPORT

# ==============================================================================
# 0. PŘEDPOKLADY - ZDE PŘIŘAĎTE VAŠE NATRÉNOVANÉ MODELY
# ==============================================================================
# POZNÁMKA: Sítě trained_model_bkn a trained_model_knetR musí být␣
↪inicializovány
# s objektem `sys_model` (chybná dynamika) před spuštěním tohoto skriptu!
try:
    trained_model_bkn = trained_model
    trained_model_knetR = state_knetR
    print("INFO: Všechny natrénované modely nalezeny a přiřazeny.")
except NameError:
    print("VAROVÁNÍ: Některé z proměnných `trained_model`, nebo `state_knetR`␣
↪nebyly nalezeny.")
    print("         Ujistěte se, že jste nejprve úspěšně dokončili trénink␣
↪všech modelů.")

# ==============================================================================
# 1. KONFIGURACE TESTU
# ==============================================================================
# POZOR: Nastaveno na 200, aby odpovídalo délce tréninku a síť nešla do OOD
TEST_SEQ_LEN = 500
```

```python
NUM_TEST_TRAJ = 100
J_SAMPLES_TEST = 100

# ==============================================================================
# 2. PŘÍPRAVA DAT (Generováno z ideálního / skutečného světa)
# ==============================================================================
print(f"\nGeneruji {NUM_TEST_TRAJ} testovacích trajektorií o délce␣
↪{TEST_SEQ_LEN}...")
# Data generujeme VŽDY pomocí sys_true (skutečná fyzika)
x_test, y_test = utils.generate_data(sys_true, num_trajectories=NUM_TEST_TRAJ,␣
↪seq_len=TEST_SEQ_LEN)
test_dataset = TensorDataset(x_test, y_test)
test_loader = DataLoader(test_dataset, batch_size=1, shuffle=False)
print("Generování dat dokončeno.")

# ==============================================================================
# 3. INICIALIZACE STATICKÝCH FILTRŮ (TEST NA NEPŘESNÉM MODELU)
# ==============================================================================
# Každý filtr dostane svou vlastní odizolovanou kopii systému
kf_mismatched = Filters.KalmanFilter(deepcopy(sys_model))
ekf_mismatched = Filters.ExtendedKalmanFilter(deepcopy(sys_model))
ukf_mismatched = Filters.UnscentedKalmanFilter(deepcopy(sys_model))

# Ponecháme si jeden čistě ideální KF jako referenci (Spodní mez chyby)
kf_ideal = Filters.KalmanFilter(deepcopy(sys_true))

print("Statické filtry inicializovány s oddělenými kopiemi NEPŘESNÉHO modelu␣
↪(sys_model).")
print("Ideální KF inicializován s PŘESNÝM modelem (sys_true) jako benchmark.")

# ==============================================================================
# 4. VYHODNOCOVACÍ SMYČKA
# ==============================================================================
all_x_true_cpu = []
all_x_hat_bkn_cpu, all_P_hat_bkn_cpu = [], []
all_x_hat_knetR_cpu, all_P_hat_knetR_cpu = [], []
all_x_hat_kf_m_cpu, all_P_hat_kf_m_cpu = [], []
all_x_hat_ekf_m_cpu, all_P_hat_ekf_m_cpu = [], []
all_x_hat_ukf_m_cpu, all_P_hat_ukf_m_cpu = [], []
all_x_hat_akf_mdm_m_cpu, all_P_hat_akf_mdm_m_cpu = [], []
all_x_hat_akf_mehra_m_cpu = []
all_x_hat_kf_ideal_cpu, all_P_hat_kf_ideal_cpu = [], []

print(f"\nVyhodnocuji modely na {NUM_TEST_TRAJ} testovacích trajektoriích...")

trained_model_bkn.eval()
trained_model_knetR.eval()
```

```
with torch.no_grad():
    for i, (x_true_seq_batch, y_test_seq_batch) in enumerate(test_loader):
        y_test_seq_gpu = y_test_seq_batch.squeeze(0).to(device)
        x_true_seq_gpu = x_true_seq_batch.squeeze(0).to(device)
        initial_state = x_true_seq_gpu[0, :].unsqueeze(0)

        # --- A. Bayesian KalmanNet (Trajectory-wise) ---
        ensemble_trajectories = []
        for j in range(J_SAMPLES_TEST):
            trained_model_bkn.reset(batch_size=1, initial_state=initial_state)
            current_x_hats = []
            for t in range(1, TEST_SEQ_LEN):
                x_filtered_t, _ = trained_model_bkn.step(y_test_seq_gpu[t, :].
 ↪unsqueeze(0))
                current_x_hats.append(x_filtered_t)
            ensemble_trajectories.append(torch.cat(current_x_hats, dim=0))
        ensemble = torch.stack(ensemble_trajectories, dim=0)
        predictions_bkn = ensemble.mean(dim=0)
        diff = ensemble - predictions_bkn.unsqueeze(0)
        covariances_bkn = (diff.unsqueeze(-1) @ diff.unsqueeze(-2)).mean(dim=0)
        full_x_hat_bkn = torch.cat([initial_state, predictions_bkn], dim=0)
        # AI startuje z chybného P0
        full_P_hat_bkn = torch.cat([sys_model.P0.unsqueeze(0),␣
 ↪covariances_bkn], dim=0)
        print(f"Trajektorie {i + 1}/{NUM_TEST_TRAJ}: Bayesian KNet (BKN)␣
 ↪dokončen.")

        # --- B. StateKalmanNetWithKnownR ---
        trained_model_knetR.reset(batch_size=1, initial_state=initial_state)
        knetR_preds_x, knetR_preds_P = [], []
        for t in range(1, TEST_SEQ_LEN):
            x_filtered_t, P_filtered_t = trained_model_knetR.
 ↪step(y_test_seq_gpu[t, :].unsqueeze(0))
            knetR_preds_x.append(x_filtered_t)
            knetR_preds_P.append(P_filtered_t)
        full_x_hat_knetR = torch.cat([initial_state, torch.cat(knetR_preds_x,␣
 ↪dim=0)], dim=0)

        processed_P_list = []
        for p_tensor in knetR_preds_P:
            while p_tensor.dim() < 2: p_tensor = p_tensor.unsqueeze(-1)
            if p_tensor.dim() > 2 and p_tensor.shape[0] == 1: p_tensor =␣
 ↪p_tensor.squeeze(0)
            processed_P_list.append(p_tensor)
```

```
        P_sequence_knetR = torch.stack(processed_P_list, dim=0)
        P0_for_cat = sys_model.P0.clone()
        while P0_for_cat.dim() < P_sequence_knetR.dim(): P0_for_cat =␣
↪P0_for_cat.unsqueeze(0)
        full_P_hat_knetR = torch.cat([P0_for_cat, P_sequence_knetR], dim=0)
        print(f"Trajektorie {i + 1}/{NUM_TEST_TRAJ}: StateKalmanNetWithKnownR␣
↪(KNetR) dokončen.")

        # --- C. Klasické filtry (MISMATCHED) ---
        kf_m_res = kf_mismatched.process_sequence(y_test_seq_gpu, Ex0=sys_model.
↪Ex0, P0=sys_model.P0)
        full_x_hat_kf_m = kf_m_res['x_filtered']
        full_P_hat_kf_m = kf_m_res['P_filtered']

        ekf_m_res = ekf_mismatched.process_sequence(y_test_seq_gpu,␣
↪Ex0=sys_model.Ex0, P0=sys_model.P0)
        full_x_hat_ekf_m = ekf_m_res['x_filtered']
        full_P_hat_ekf_m = ekf_m_res['P_filtered']

        ukf_m_res = ukf_mismatched.process_sequence(y_test_seq_gpu,␣
↪Ex0=sys_model.Ex0, P0=sys_model.P0)
        full_x_hat_ukf_m = ukf_m_res['x_filtered']
        full_P_hat_ukf_m = ukf_m_res['P_filtered']

        # Tímto zabráníme přenosu poškozené matice Q mezi trajektoriemi
        akf_mdm = Filters.AdaptiveKalmanFilter_default(deepcopy(sys_model),␣
↪mdm_L=5, mdm_version=2, lambda_rls=1.0, init_sigma_rls_value=0.1)
        akf_mehra = Filters.
↪AdaptiveKalmanFilter_mehra_offline(deepcopy(sys_model), num_iterations=5)

        afk_mdm_m_res,_,_ = akf_mdm.process_sequence_adaptively(y_test_seq_gpu)
        full_x_hat_akf_mdm_m = afk_mdm_m_res['x_filtered']
        full_P_hat_akf_mdm_m = afk_mdm_m_res['P_filtered']

        akf_mehra_m_res = akf_mehra.process_sequence(y_test_seq_gpu)
        full_x_hat_akf_mehra_m = akf_mehra_m_res['x_filtered']
        print(f"Trajektorie {i + 1}/{NUM_TEST_TRAJ}: Mismatched model-based␣
↪filtry dokončeny.")

        # --- E. Benchmark (IDEAL) ---
        kf_i_res = kf_ideal.process_sequence(y_test_seq_gpu, Ex0=sys_true.Ex0,␣
↪P0=sys_true.P0)
        full_x_hat_kf_ideal = kf_i_res['x_filtered']
        full_P_hat_kf_ideal = kf_i_res['P_filtered']
        print(f"Trajektorie {i + 1}/{NUM_TEST_TRAJ}: KF (Ideální Benchmark)␣
↪dokončen.")
```

```
        # Ukládání na CPU
        all_x_true_cpu.append(x_true_seq_gpu.cpu())
        all_x_hat_bkn_cpu.append(full_x_hat_bkn.cpu()); all_P_hat_bkn_cpu.
↪append(full_P_hat_bkn.cpu())
        all_x_hat_knetR_cpu.append(full_x_hat_knetR.cpu()); all_P_hat_knetR_cpu.
↪append(full_P_hat_knetR.cpu())

        all_x_hat_kf_m_cpu.append(full_x_hat_kf_m.cpu()); all_P_hat_kf_m_cpu.
↪append(full_P_hat_kf_m.cpu())
        all_x_hat_ekf_m_cpu.append(full_x_hat_ekf_m.cpu()); all_P_hat_ekf_m_cpu.
↪append(full_P_hat_ekf_m.cpu())
        all_x_hat_ukf_m_cpu.append(full_x_hat_ukf_m.cpu()); all_P_hat_ukf_m_cpu.
↪append(full_P_hat_ukf_m.cpu())
        all_x_hat_akf_mdm_m_cpu.append(full_x_hat_akf_mdm_m.cpu());␣
↪all_P_hat_akf_mdm_m_cpu.append(full_P_hat_akf_mdm_m.cpu())
        all_x_hat_akf_mehra_m_cpu.append(full_x_hat_akf_mehra_m.cpu())

        all_x_hat_kf_ideal_cpu.append(full_x_hat_kf_ideal.cpu());␣
↪all_P_hat_kf_ideal_cpu.append(full_P_hat_kf_ideal.cpu())

        print(f"Dokončena trajektorie {i + 1}/{NUM_TEST_TRAJ}...")

# ==============================================================================
# 5. FINÁLNÍ VÝPOČET A VÝPIS METRIK
# ==============================================================================
mse_bkn, anees_bkn = [], []; mse_knetR, anees_knetR = [], []
mse_kf_m, anees_kf_m = [], []
mse_ekf_m, anees_ekf_m = [], []
mse_ukf_m, anees_ukf_m = [], []
mse_akf_mdm_m, anees_akf_mdm_m = [], []
mse_akf_mehra_m = []
mse_kf_ideal, anees_kf_ideal = [], []

print("\nPočítám finální metriky pro jednotlivé trajektorie...")

with torch.no_grad():
    for i in range(NUM_TEST_TRAJ):
        x_true = all_x_true_cpu[i]

        def get_metrics(x_hat, P_hat):
            mse = F.mse_loss(x_hat[1:], x_true[1:]).item()
            anees = utils.calculate_anees_vectorized(x_true.unsqueeze(0), x_hat.
↪unsqueeze(0), P_hat.unsqueeze(0))
            return mse, anees
```

```
        # Data-driven
        mse, anees = get_metrics(all_x_hat_bkn_cpu[i], all_P_hat_bkn_cpu[i]);␣
↪mse_bkn.append(mse); anees_bkn.append(anees)
        mse, anees = get_metrics(all_x_hat_knetR_cpu[i],␣
↪all_P_hat_knetR_cpu[i]); mse_knetR.append(mse); anees_knetR.append(anees)

        # Mismatched model-based
        mse, anees = get_metrics(all_x_hat_kf_m_cpu[i], all_P_hat_kf_m_cpu[i]);␣
↪mse_kf_m.append(mse); anees_kf_m.append(anees)
        mse, anees = get_metrics(all_x_hat_ekf_m_cpu[i],␣
↪all_P_hat_ekf_m_cpu[i]); mse_ekf_m.append(mse); anees_ekf_m.append(anees)
        mse, anees = get_metrics(all_x_hat_ukf_m_cpu[i],␣
↪all_P_hat_ukf_m_cpu[i]); mse_ukf_m.append(mse); anees_ukf_m.append(anees)
        mse, anees = get_metrics(all_x_hat_akf_mdm_m_cpu[i],␣
↪all_P_hat_akf_mdm_m_cpu[i]); mse_akf_mdm_m.append(mse); anees_akf_mdm_m.
↪append(anees)
        mse_akf_mehra_m.append(F.mse_loss(all_x_hat_akf_mehra_m_cpu[i][1:],␣
↪x_true[1:]).item())

        # Benchmark ideal
        mse, anees = get_metrics(all_x_hat_kf_ideal_cpu[i],␣
↪all_P_hat_kf_ideal_cpu[i]); mse_kf_ideal.append(mse); anees_kf_ideal.
↪append(anees)

def avg(metric_list): return np.mean([m for m in metric_list if not np.
↪isnan(m)])
state_dim_for_nees = all_x_true_cpu[0].shape[1]

# --- Finální výpis tabulky ---
print("\n" + "="*85)
print(f"FINÁLNÍ VÝSLEDKY: MODEL MISMATCH SCÉNÁŘ (průměr přes {NUM_TEST_TRAJ}␣
↪běhů)")
print("Filtrům byla dodána chybná rotace F_model a chybný šum Q_model.")
print("="*85)
print(f"{'Model':<35} | {'Průměrné MSE':<20} | {'Průměrný ANEES':<20}")
print("-" * 85)
print(f"{'--- AI Filtry (Trénováno na chybě) ---':<35} | {'(nižší je lepší)':
↪<20} | {'(bližší ' + str(float(state_dim_for_nees)) + ' je lepší)':<20}")
print(f"{'Bayesian KNet (BKN)':<35} | {avg(mse_bkn):<20.4f} | {avg(anees_bkn):
↪<20.4f}")
print(f"{'KNet with Known R (KNetR)':<35} | {avg(mse_knetR):<20.4f} |␣
↪{avg(anees_knetR):<20.4f}")
print("-" * 85)
print(f"{'--- Klasické Filtry (Chybný model) ---':<35} | {'':<20} | {'':<20}")
print(f"{'KF (Nepřesný model)':<35} | {avg(mse_kf_m):<20.4f} | {avg(anees_kf_m):
↪<20.4f}")
```

```
print(f"{'EKF (Nepřesný model)':<35} | {avg(mse_ekf_m):<20.4f} |␣
↪{avg(anees_ekf_m):<20.4f}")
print(f"{'UKF (Nepřesný model)':<35} | {avg(mse_ukf_m):<20.4f} |␣
↪{avg(anees_ukf_m):<20.4f}")
print(f"{'AKF-MDM (Nepřesný model)':<35} | {avg(mse_akf_mdm_m):<20.4f} |␣
↪{avg(anees_akf_mdm_m):<20.4f}")
print(f"{'AKF-Mehra (Nepřesný model)':<35} | {avg(mse_akf_mehra_m):<20.4f} |␣
↪{'N/A':<20}")
print("-" * 85)
print(f"{'--- BENCHMARK (Fyzikální limit) ---':<35} | {'':<20} | {'':<20}")
print(f"{'KF (Ideální model - OPTIMUM)':<35} | {avg(mse_kf_ideal):<20.4f} |␣
↪{avg(anees_kf_ideal):<20.4f}")
print("="*85)
```

```
INFO: Všechny natrénované modely nalezeny a přiřazeny.

Generuji 100 testovacích trajektorií o délce 500…
Generování dat dokončeno.
Statické filtry inicializovány s oddělenými kopiemi NEPŘESNÉHO modelu
(sys_model).
Ideální KF inicializován s PŘESNÝM modelem (sys_true) jako benchmark.

Vyhodnocuji modely na 100 testovacích trajektoriích…
Trajektorie 1/100: Bayesian KNet (BKN) dokončen.
Trajektorie 1/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
759.59it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000



Trajektorie 1/100: Mismatched model-based filtry dokončeny.



Trajektorie 1/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 1/100…
Trajektorie 2/100: Bayesian KNet (BKN) dokončen.
Trajektorie 2/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
770.87it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000
```

```
Trajektorie 2/100: Mismatched model-based filtry dokončeny.


Trajektorie 2/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 2/100…
Trajektorie 3/100: Bayesian KNet (BKN) dokončen.
Trajektorie 3/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
819.99it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 3/100: Mismatched model-based filtry dokončeny.


Trajektorie 3/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 3/100…
Trajektorie 4/100: Bayesian KNet (BKN) dokončen.
Trajektorie 4/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
783.45it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 4/100: Mismatched model-based filtry dokončeny.


Trajektorie 4/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 4/100…
Trajektorie 5/100: Bayesian KNet (BKN) dokončen.
Trajektorie 5/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
818.15it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 5/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 5/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 5/100…
Trajektorie 6/100: Bayesian KNet (BKN) dokončen.
Trajektorie 6/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
780.77it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 6/100: Mismatched model-based filtry dokončeny.


Trajektorie 6/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 6/100…


Trajektorie 7/100: Bayesian KNet (BKN) dokončen.
Trajektorie 7/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
754.53it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 7/100: Mismatched model-based filtry dokončeny.


Trajektorie 7/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 7/100…


Trajektorie 8/100: Bayesian KNet (BKN) dokončen.
Trajektorie 8/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
806.61it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000
```

```
Trajektorie 8/100: Mismatched model-based filtry dokončeny.


Trajektorie 8/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 8/100…
Trajektorie 9/100: Bayesian KNet (BKN) dokončen.
Trajektorie 9/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
814.36it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 9/100: Mismatched model-based filtry dokončeny.


Trajektorie 9/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 9/100…
Trajektorie 10/100: Bayesian KNet (BKN) dokončen.
Trajektorie 10/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
773.71it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 10/100: Mismatched model-based filtry dokončeny.


Trajektorie 10/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 10/100…
Trajektorie 11/100: Bayesian KNet (BKN) dokončen.
Trajektorie 11/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
822.72it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 11/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 11/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 11/100…
Trajektorie 12/100: Bayesian KNet (BKN) dokončen.
Trajektorie 12/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
824.77it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 12/100: Mismatched model-based filtry dokončeny.


Trajektorie 12/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 12/100…
Trajektorie 13/100: Bayesian KNet (BKN) dokončen.
Trajektorie 13/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
836.65it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 13/100: Mismatched model-based filtry dokončeny.


Trajektorie 13/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 13/100…
Trajektorie 14/100: Bayesian KNet (BKN) dokončen.
Trajektorie 14/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
816.98it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 14/100: Mismatched model-based filtry dokončeny.


Trajektorie 14/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 14/100…
```

```
Trajektorie 15/100: Bayesian KNet (BKN) dokončen.
Trajektorie 15/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
819.34it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 15/100: Mismatched model-based filtry dokončeny.


Trajektorie 15/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 15/100…
Trajektorie 16/100: Bayesian KNet (BKN) dokončen.
Trajektorie 16/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
821.52it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 16/100: Mismatched model-based filtry dokončeny.


Trajektorie 16/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 16/100…


Trajektorie 17/100: Bayesian KNet (BKN) dokončen.
Trajektorie 17/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
822.18it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 17/100: Mismatched model-based filtry dokončeny.


Trajektorie 17/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 17/100…
```

```
Trajektorie 18/100: Bayesian KNet (BKN) dokončen.
Trajektorie 18/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
808.65it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 18/100: Mismatched model-based filtry dokončeny.


Trajektorie 18/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 18/100…
Trajektorie 19/100: Bayesian KNet (BKN) dokončen.
Trajektorie 19/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
828.24it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 19/100: Mismatched model-based filtry dokončeny.


Trajektorie 19/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 19/100…
Trajektorie 20/100: Bayesian KNet (BKN) dokončen.
Trajektorie 20/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
825.91it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 20/100: Mismatched model-based filtry dokončeny.


Trajektorie 20/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 20/100…
```

```
Trajektorie 21/100: Bayesian KNet (BKN) dokončen.
Trajektorie 21/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
824.69it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 21/100: Mismatched model-based filtry dokončeny.


Trajektorie 21/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 21/100…
Trajektorie 22/100: Bayesian KNet (BKN) dokončen.
Trajektorie 22/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
814.64it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 22/100: Mismatched model-based filtry dokončeny.


Trajektorie 22/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 22/100…


Trajektorie 23/100: Bayesian KNet (BKN) dokončen.
Trajektorie 23/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
814.13it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 23/100: Mismatched model-based filtry dokončeny.


Trajektorie 23/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 23/100…
```

```
Trajektorie 24/100: Bayesian KNet (BKN) dokončen.
Trajektorie 24/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
814.09it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 24/100: Mismatched model-based filtry dokončeny.


Trajektorie 24/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 24/100…
Trajektorie 25/100: Bayesian KNet (BKN) dokončen.
Trajektorie 25/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
832.89it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 25/100: Mismatched model-based filtry dokončeny.


Trajektorie 25/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 25/100…
Trajektorie 26/100: Bayesian KNet (BKN) dokončen.
Trajektorie 26/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
774.85it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 26/100: Mismatched model-based filtry dokončeny.


Trajektorie 26/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 26/100…
```

```
Trajektorie 27/100: Bayesian KNet (BKN) dokončen.
Trajektorie 27/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
833.28it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 27/100: Mismatched model-based filtry dokončeny.


Trajektorie 27/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 27/100…


Trajektorie 28/100: Bayesian KNet (BKN) dokončen.
Trajektorie 28/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
822.11it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 28/100: Mismatched model-based filtry dokončeny.


Trajektorie 28/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 28/100…
Trajektorie 29/100: Bayesian KNet (BKN) dokončen.
Trajektorie 29/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
823.44it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 29/100: Mismatched model-based filtry dokončeny.


Trajektorie 29/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 29/100…
```

```
Trajektorie 30/100: Bayesian KNet (BKN) dokončen.
Trajektorie 30/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
822.73it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 30/100: Mismatched model-based filtry dokončeny.


Trajektorie 30/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 30/100…
Trajektorie 31/100: Bayesian KNet (BKN) dokončen.
Trajektorie 31/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
812.05it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 31/100: Mismatched model-based filtry dokončeny.


Trajektorie 31/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 31/100…


Trajektorie 32/100: Bayesian KNet (BKN) dokončen.
Trajektorie 32/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
794.56it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 32/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 32/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 32/100…
Trajektorie 33/100: Bayesian KNet (BKN) dokončen.
Trajektorie 33/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
822.50it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 33/100: Mismatched model-based filtry dokončeny.


Trajektorie 33/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 33/100…
Trajektorie 34/100: Bayesian KNet (BKN) dokončen.
Trajektorie 34/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
826.89it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 34/100: Mismatched model-based filtry dokončeny.


Trajektorie 34/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 34/100…


Trajektorie 35/100: Bayesian KNet (BKN) dokončen.
Trajektorie 35/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
821.72it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 35/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 35/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 35/100…
Trajektorie 36/100: Bayesian KNet (BKN) dokončen.
Trajektorie 36/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
816.52it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 36/100: Mismatched model-based filtry dokončeny.


Trajektorie 36/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 36/100…
Trajektorie 37/100: Bayesian KNet (BKN) dokončen.
Trajektorie 37/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
751.47it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 37/100: Mismatched model-based filtry dokončeny.


Trajektorie 37/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 37/100…
Trajektorie 38/100: Bayesian KNet (BKN) dokončen.
Trajektorie 38/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
827.66it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 38/100: Mismatched model-based filtry dokončeny.


Trajektorie 38/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 38/100…
```

```
Trajektorie 39/100: Bayesian KNet (BKN) dokončen.
Trajektorie 39/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
827.33it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 39/100: Mismatched model-based filtry dokončeny.


Trajektorie 39/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 39/100…
Trajektorie 40/100: Bayesian KNet (BKN) dokončen.
Trajektorie 40/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
814.15it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 40/100: Mismatched model-based filtry dokončeny.


Trajektorie 40/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 40/100…
Trajektorie 41/100: Bayesian KNet (BKN) dokončen.
Trajektorie 41/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
817.23it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 41/100: Mismatched model-based filtry dokončeny.


Trajektorie 41/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 41/100…
Trajektorie 42/100: Bayesian KNet (BKN) dokončen.
Trajektorie 42/100: StateKalmanNetWithKnownR (KNetR) dokončen.
```

```
Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
824.24it/s]

--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 42/100: Mismatched model-based filtry dokončeny.


Trajektorie 42/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 42/100…


Trajektorie 43/100: Bayesian KNet (BKN) dokončen.
Trajektorie 43/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
829.10it/s]

--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 43/100: Mismatched model-based filtry dokončeny.


Trajektorie 43/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 43/100…
Trajektorie 44/100: Bayesian KNet (BKN) dokončen.
Trajektorie 44/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
819.40it/s]

--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 44/100: Mismatched model-based filtry dokončeny.


Trajektorie 44/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 44/100…
```

```
Trajektorie 45/100: Bayesian KNet (BKN) dokončen.
Trajektorie 45/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
820.53it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 45/100: Mismatched model-based filtry dokončeny.


Trajektorie 45/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 45/100…


Trajektorie 46/100: Bayesian KNet (BKN) dokončen.
Trajektorie 46/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
820.54it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 46/100: Mismatched model-based filtry dokončeny.


Trajektorie 46/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 46/100…
Trajektorie 47/100: Bayesian KNet (BKN) dokončen.
Trajektorie 47/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
821.65it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 47/100: Mismatched model-based filtry dokončeny.


Trajektorie 47/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 47/100…
```

```
Trajektorie 48/100: Bayesian KNet (BKN) dokončen.
Trajektorie 48/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
737.29it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 48/100: Mismatched model-based filtry dokončeny.


Trajektorie 48/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 48/100…
Trajektorie 49/100: Bayesian KNet (BKN) dokončen.
Trajektorie 49/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
730.41it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 49/100: Mismatched model-based filtry dokončeny.


Trajektorie 49/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 49/100…


Trajektorie 50/100: Bayesian KNet (BKN) dokončen.
Trajektorie 50/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
818.73it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 50/100: Mismatched model-based filtry dokončeny.


Trajektorie 50/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 50/100…
```

```
Trajektorie 51/100: Bayesian KNet (BKN) dokončen.
Trajektorie 51/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
809.18it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 51/100: Mismatched model-based filtry dokončeny.


Trajektorie 51/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 51/100…


Trajektorie 52/100: Bayesian KNet (BKN) dokončen.
Trajektorie 52/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
807.39it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 52/100: Mismatched model-based filtry dokončeny.


Trajektorie 52/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 52/100…


Trajektorie 53/100: Bayesian KNet (BKN) dokončen.
Trajektorie 53/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
806.94it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 53/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 53/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 53/100…


Trajektorie 54/100: Bayesian KNet (BKN) dokončen.
Trajektorie 54/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
817.70it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 54/100: Mismatched model-based filtry dokončeny.


Trajektorie 54/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 54/100…
Trajektorie 55/100: Bayesian KNet (BKN) dokončen.
Trajektorie 55/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
790.20it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 55/100: Mismatched model-based filtry dokončeny.


Trajektorie 55/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 55/100…
Trajektorie 56/100: Bayesian KNet (BKN) dokončen.
Trajektorie 56/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
823.71it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 56/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 56/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 56/100…


Trajektorie 57/100: Bayesian KNet (BKN) dokončen.
Trajektorie 57/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
837.33it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 57/100: Mismatched model-based filtry dokončeny.


Trajektorie 57/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 57/100…
Trajektorie 58/100: Bayesian KNet (BKN) dokončen.
Trajektorie 58/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
797.58it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 58/100: Mismatched model-based filtry dokončeny.


Trajektorie 58/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 58/100…


Trajektorie 59/100: Bayesian KNet (BKN) dokončen.
Trajektorie 59/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
822.61it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 59/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 59/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 59/100…


Trajektorie 60/100: Bayesian KNet (BKN) dokončen.
Trajektorie 60/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
823.31it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 60/100: Mismatched model-based filtry dokončeny.


Trajektorie 60/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 60/100…


Trajektorie 61/100: Bayesian KNet (BKN) dokončen.
Trajektorie 61/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
804.85it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 61/100: Mismatched model-based filtry dokončeny.


Trajektorie 61/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 61/100…


Trajektorie 62/100: Bayesian KNet (BKN) dokončen.
Trajektorie 62/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
808.26it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000
```

```
Trajektorie 62/100: Mismatched model-based filtry dokončeny.


Trajektorie 62/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 62/100…


Trajektorie 63/100: Bayesian KNet (BKN) dokončen.
Trajektorie 63/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
826.85it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 63/100: Mismatched model-based filtry dokončeny.


Trajektorie 63/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 63/100…
Trajektorie 64/100: Bayesian KNet (BKN) dokončen.
Trajektorie 64/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
786.29it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 64/100: Mismatched model-based filtry dokončeny.


Trajektorie 64/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 64/100…
Trajektorie 65/100: Bayesian KNet (BKN) dokončen.
Trajektorie 65/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
817.15it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000
```

```
Trajektorie 65/100: Mismatched model-based filtry dokončeny.


Trajektorie 65/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 65/100…


Trajektorie 66/100: Bayesian KNet (BKN) dokončen.
Trajektorie 66/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
821.98it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 66/100: Mismatched model-based filtry dokončeny.


Trajektorie 66/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 66/100…


Trajektorie 67/100: Bayesian KNet (BKN) dokončen.
Trajektorie 67/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
813.70it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 67/100: Mismatched model-based filtry dokončeny.


Trajektorie 67/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 67/100…


Trajektorie 68/100: Bayesian KNet (BKN) dokončen.
Trajektorie 68/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
826.41it/s]
```

```
--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 68/100: Mismatched model-based filtry dokončeny.


Trajektorie 68/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 68/100…


Trajektorie 69/100: Bayesian KNet (BKN) dokončen.
Trajektorie 69/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
808.94it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 69/100: Mismatched model-based filtry dokončeny.


Trajektorie 69/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 69/100…
Trajektorie 70/100: Bayesian KNet (BKN) dokončen.
Trajektorie 70/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
829.04it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 70/100: Mismatched model-based filtry dokončeny.


Trajektorie 70/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 70/100…
Trajektorie 71/100: Bayesian KNet (BKN) dokončen.
Trajektorie 71/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
813.74it/s]
```

```
--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 71/100: Mismatched model-based filtry dokončeny.


Trajektorie 71/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 71/100…
Trajektorie 72/100: Bayesian KNet (BKN) dokončen.
Trajektorie 72/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
786.69it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 72/100: Mismatched model-based filtry dokončeny.


Trajektorie 72/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 72/100…
Trajektorie 73/100: Bayesian KNet (BKN) dokončen.
Trajektorie 73/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
793.23it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 73/100: Mismatched model-based filtry dokončeny.


Trajektorie 73/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 73/100…
Trajektorie 74/100: Bayesian KNet (BKN) dokončen.
Trajektorie 74/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
820.02it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000
```

```
Trajektorie 74/100: Mismatched model-based filtry dokončeny.


Trajektorie 74/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 74/100…
Trajektorie 75/100: Bayesian KNet (BKN) dokončen.
Trajektorie 75/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
809.20it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 75/100: Mismatched model-based filtry dokončeny.


Trajektorie 75/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 75/100…
Trajektorie 76/100: Bayesian KNet (BKN) dokončen.
Trajektorie 76/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
808.87it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 76/100: Mismatched model-based filtry dokončeny.


Trajektorie 76/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 76/100…
Trajektorie 77/100: Bayesian KNet (BKN) dokončen.
Trajektorie 77/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
816.30it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 77/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 77/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 77/100…


Trajektorie 78/100: Bayesian KNet (BKN) dokončen.
Trajektorie 78/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
826.84it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 78/100: Mismatched model-based filtry dokončeny.


Trajektorie 78/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 78/100…
Trajektorie 79/100: Bayesian KNet (BKN) dokončen.
Trajektorie 79/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
829.98it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 79/100: Mismatched model-based filtry dokončeny.


Trajektorie 79/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 79/100…
Trajektorie 80/100: Bayesian KNet (BKN) dokončen.
Trajektorie 80/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
828.02it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 80/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 80/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 80/100…


Trajektorie 81/100: Bayesian KNet (BKN) dokončen.
Trajektorie 81/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
828.58it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 81/100: Mismatched model-based filtry dokončeny.


Trajektorie 81/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 81/100…
Trajektorie 82/100: Bayesian KNet (BKN) dokončen.
Trajektorie 82/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
829.16it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 82/100: Mismatched model-based filtry dokončeny.


Trajektorie 82/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 82/100…
Trajektorie 83/100: Bayesian KNet (BKN) dokončen.
Trajektorie 83/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
861.09it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 83/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 83/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 83/100…


Trajektorie 84/100: Bayesian KNet (BKN) dokončen.
Trajektorie 84/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
724.10it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 84/100: Mismatched model-based filtry dokončeny.


Trajektorie 84/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 84/100…
Trajektorie 85/100: Bayesian KNet (BKN) dokončen.
Trajektorie 85/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
788.91it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 85/100: Mismatched model-based filtry dokončeny.


Trajektorie 85/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 85/100…
Trajektorie 86/100: Bayesian KNet (BKN) dokončen.
Trajektorie 86/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
769.74it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 86/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 86/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 86/100…
Trajektorie 87/100: Bayesian KNet (BKN) dokončen.
Trajektorie 87/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
754.66it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 87/100: Mismatched model-based filtry dokončeny.


Trajektorie 87/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 87/100…
Trajektorie 88/100: Bayesian KNet (BKN) dokončen.
Trajektorie 88/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
772.38it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 88/100: Mismatched model-based filtry dokončeny.


Trajektorie 88/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 88/100…


Trajektorie 89/100: Bayesian KNet (BKN) dokončen.
Trajektorie 89/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
751.44it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 89/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 89/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 89/100…
Trajektorie 90/100: Bayesian KNet (BKN) dokončen.
Trajektorie 90/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
701.08it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 90/100: Mismatched model-based filtry dokončeny.


Trajektorie 90/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 90/100…


Trajektorie 91/100: Bayesian KNet (BKN) dokončen.
Trajektorie 91/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
756.91it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 91/100: Mismatched model-based filtry dokončeny.


Trajektorie 91/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 91/100…


Trajektorie 92/100: Bayesian KNet (BKN) dokončen.
Trajektorie 92/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
742.96it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 92/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 92/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 92/100…


Trajektorie 93/100: Bayesian KNet (BKN) dokončen.
Trajektorie 93/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
742.50it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 93/100: Mismatched model-based filtry dokončeny.


Trajektorie 93/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 93/100…
Trajektorie 94/100: Bayesian KNet (BKN) dokončen.
Trajektorie 94/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
751.51it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 94/100: Mismatched model-based filtry dokončeny.


Trajektorie 94/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 94/100…
Trajektorie 95/100: Bayesian KNet (BKN) dokončen.
Trajektorie 95/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|      | 500/500 [00:00<00:00,
758.74it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 95/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 95/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 95/100…


Trajektorie 96/100: Bayesian KNet (BKN) dokončen.
Trajektorie 96/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
702.48it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 96/100: Mismatched model-based filtry dokončeny.


Trajektorie 96/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 96/100…


Trajektorie 97/100: Bayesian KNet (BKN) dokončen.
Trajektorie 97/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
811.00it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 97/100: Mismatched model-based filtry dokončeny.


Trajektorie 97/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 97/100…
Trajektorie 98/100: Bayesian KNet (BKN) dokončen.
Trajektorie 98/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
831.64it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 98/100: Mismatched model-based filtry dokončeny.
```

```
Trajektorie 98/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 98/100…
Trajektorie 99/100: Bayesian KNet (BKN) dokončen.
Trajektorie 99/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
726.25it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 99/100: Mismatched model-based filtry dokončeny.


Trajektorie 99/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 99/100…


Trajektorie 100/100: Bayesian KNet (BKN) dokončen.
Trajektorie 100/100: StateKalmanNetWithKnownR (KNetR) dokončen.

Processing sequence adaptively: 100%|     | 500/500 [00:00<00:00,
740.26it/s]


--- Offline Mehra: Start adaptace ---
Iterace 0 (Nástřel): K[0,0] = 0.1000


Trajektorie 100/100: Mismatched model-based filtry dokončeny.


Trajektorie 100/100: KF (Ideální Benchmark) dokončen.
Dokončena trajektorie 100/100…

Počítám finální metriky pro jednotlivé trajektorie…



================================================================================
=====
FINÁLNÍ VÝSLEDKY: MODEL MISMATCH SCÉNÁŘ (průměr přes 100 běhů)
Filtrům byla dodána chybná rotace F_model a chybný šum Q_model.
================================================================================
=====
Model                                | Průměrné MSE         | Průměrný ANEES
```

```
--------------------------------------------------------------------------------
-----
--- AI Filtry (Trénováno na chybě) --- | (nižší je lepší)     | (bližší 2.0 je
lepší)
Bayesian KNet (BKN)                    | 1.3347               | 2.1145
KNet with Known R (KNetR)              | 0.8795               | 3.4001
--------------------------------------------------------------------------------
-----
--- Klasické Filtry (Chybný model) --- |                      |
KF (Nepřesný model)                    | 6.7890               | 33.8727
EKF (Nepřesný model)                   | 6.7890               | 33.8727
UKF (Nepřesný model)                   | 6.7890               | 33.8727
AKF-MDM (Nepřesný model)               | 1.3895               | 36579.6125
AKF-Mehra (Nepřesný model)             | 1.1612               | N/A
--------------------------------------------------------------------------------
-----
--- BENCHMARK (Fyzikální limit) --- |                      |
KF (Ideální model - OPTIMUM)           | 0.4055               | 2.0205
================================================================================
=====
```

1 Kalman Gain comparison

[10]:
```python
import matplotlib.pyplot as plt
import torch
import numpy as np

# ==============================================================================
# ANALÝZA VÝVOJE KALMANOVA ZISKU A POROVNÁNÍ VŮČI OPTIMU (FROBENIOVA NORMA)
# SCÉNÁŘ: MODEL MISMATCH
# ==============================================================================
num_plot_traj = min(3, NUM_TEST_TRAJ)

print("Předpočítávám výsledky filtrů a extrahuji Kalmanovy zisky pro vykreslení.
 ↪..")

# Seznamy pro uložení historie chyb (Frobeniova norma)
frob_errors_kf_m = []
frob_errors_bkn = []
frob_errors_knetR = []
frob_errors_mdm = []
frob_errors_mehra = []

trained_model_bkn.eval()
trained_model_knetR.eval()

with torch.no_grad():
```

```
    # Procházíme testovací trajektorie
    for i, (x_true_seq_batch, y_test_seq_batch) in enumerate(test_loader):
        if i >= num_plot_traj:
            break

        y_test_seq_gpu = y_test_seq_batch.squeeze(0).to(device)
        x_true_seq_gpu = x_true_seq_batch.squeeze(0).to(device)
        initial_state = x_true_seq_gpu[0, :].unsqueeze(0)
        seq_len = y_test_seq_gpu.shape[0]

        # --- 1. Ideální KF (Benchmark - Absolutní Optimum) ---
        res_ideal = kf_ideal.process_sequence(y_test_seq_gpu, Ex0=sys_true.Ex0,␣
↪P0=sys_true.P0)
        K_kf_ideal = res_ideal['Kalman_gain'].cpu()  # Očekávaný tvar:␣
↪[seq_len, state_dim, obs_dim]

        # --- 2. Nepřesný KF (Referenční spodní hranice inteligence) ---
        res_kf_m = kf_mismatched.process_sequence(y_test_seq_gpu, Ex0=sys_model.
↪Ex0, P0=sys_model.P0)
        K_kf_m = res_kf_m['Kalman_gain'].cpu()

        # --- 3. AKF MDM (Nepřesný model) ---
        res_mdm,_,_ = akf_mdm_mismatched.
↪process_sequence_adaptively(y_test_seq_gpu)
        K_mdm = res_mdm['Kalman_gain'].cpu()

        # --- 4. AKF Mehra (Nepřesný model) ---
        res_mehra = akf_mehra_mismatched.process_sequence(y_test_seq_gpu)
        K_mehra = res_mehra['Kalman_gain'].cpu()

        # Ošetření: Pokud Mehra vrací konstantní zisk pro celou trajektorii,␣
↪expandujeme ho
        if K_mehra.dim() == 2:
            K_mehra = K_mehra.unsqueeze(0).expand(seq_len, -1, -1)
        elif K_mehra.shape[0] < seq_len:
            pad_len = seq_len - K_mehra.shape[0]
            K_mehra = torch.cat([K_kf_ideal[:pad_len], K_mehra], dim=0)

        # --- 5. Bayesian KalmanNet (Zprůměrovaný zisk přes J vzorků) ---
        K_bkn_ensemble = []
        for j in range(J_SAMPLES_TEST):
            trained_model_bkn.reset(batch_size=1, initial_state=initial_state)
            for t in range(1, seq_len):
                trained_model_bkn.step(y_test_seq_gpu[t, :].unsqueeze(0))

            K_history = trained_model_bkn.get_kalman_gain_history().squeeze(1).
↪cpu()
```

```
            # AI aktualizuje zisk až od t=1, nultý krok doplníme z nepřesného␣
↪PO (získaného přes K_kf_m)
            K_history_full = torch.cat([K_kf_m[0].unsqueeze(0), K_history],␣
↪dim=0)
            K_bkn_ensemble.append(K_history_full)

        K_bkn = torch.stack(K_bkn_ensemble, dim=0).mean(dim=0)

        # --- 6. KalmanNet s Known R (KNetR) ---
        trained_model_knetR.reset(batch_size=1, initial_state=initial_state)
        for t in range(1, seq_len):
            trained_model_knetR.step(y_test_seq_gpu[t, :].unsqueeze(0))

        K_history_knetR = trained_model_knetR.get_kalman_gain_history().
↪squeeze(1).cpu()
        K_knetR = torch.cat([K_kf_m[0].unsqueeze(0), K_history_knetR], dim=0)

        # --- VÝPOČET FROBENIOVY NORMY CHYBY ||K_model - K_KF_IDEAL||_F ---
        err_kf_m = torch.linalg.matrix_norm(K_kf_m - K_kf_ideal, ord='fro',␣
↪dim=(1,2))
        err_bkn = torch.linalg.matrix_norm(K_bkn - K_kf_ideal, ord='fro',␣
↪dim=(1,2))
        err_knetR = torch.linalg.matrix_norm(K_knetR - K_kf_ideal, ord='fro',␣
↪dim=(1,2))
        err_mdm = torch.linalg.matrix_norm(K_mdm - K_kf_ideal, ord='fro',␣
↪dim=(1,2))
        err_mehra = torch.linalg.matrix_norm(K_mehra - K_kf_ideal, ord='fro',␣
↪dim=(1,2))

        frob_errors_kf_m.append(err_kf_m)
        frob_errors_bkn.append(err_bkn)
        frob_errors_knetR.append(err_knetR)
        frob_errors_mdm.append(err_mdm)
        frob_errors_mehra.append(err_mehra)

# ==============================================================================
# GRAFICKÁ A NUMERICKÁ PREZENTACE VÝSLEDKŮ
# ==============================================================================
print("\nGeneruji grafy konvergence Frobeniovy normy pro scénář Mismatch...")

fig, axes = plt.subplots(num_plot_traj, 1, figsize=(12, 5 * num_plot_traj),␣
↪sharex=True)
if num_plot_traj == 1: axes = [axes]

steady_state_ratio = 0.2
idx_steady = int((1 - steady_state_ratio) * TEST_SEQ_LEN)
```

```
num_kf_m_ss, num_bkn_ss, num_knetR_ss, num_mdm_ss, num_mehra_ss = [], [], [],␣
↪[], []

for i in range(num_plot_traj):
    ax = axes[i]

    # Referenční úroveň chyby, pokud by systém vůbec neadaptoval
    ax.plot(frob_errors_kf_m[i].numpy(), label='Statický KF (Chybný model)',␣
↪color='gray', linestyle='-.', linewidth=2, alpha=0.7)

    # Vykreslení adaptivních a AI filtrů
    ax.plot(frob_errors_mdm[i].numpy(), label='AKF (MDM)', color='green',␣
↪linewidth=1.5, alpha=0.8)
    ax.plot(frob_errors_mehra[i].numpy(), label='AKF (Mehra)', color='red',␣
↪linewidth=1.5, alpha=0.8)
    ax.plot(frob_errors_bkn[i].numpy(), label='Bayesian KNet (BKN)',␣
↪color='blue', linewidth=1.8)
    ax.plot(frob_errors_knetR[i].numpy(), label='KNet with Known R',␣
↪color='cyan', linestyle='--', linewidth=1.8)

    # Benchmarková ideální čára (Chyba = 0)
    ax.axhline(1e-6, color='black', linestyle=':', linewidth=2, label='Optimum␣
↪(Ideální KF = 0)')

    ax.set_title(f"Trajektorie {i+1}: Vývoj chyby Kalmanova zisku při Model␣
↪Mismatch ($||K - K_{{KF, ideal}}||_F$)")
    ax.set_ylabel("Chyba zisku (Log. měřítko)")
    ax.set_yscale('log')
    ax.grid(True, which="both", ls="--", alpha=0.5)

    if i == 0:
        ax.legend(loc='upper right')

    num_kf_m_ss.append(frob_errors_kf_m[i][idx_steady:].mean().item())
    num_bkn_ss.append(frob_errors_bkn[i][idx_steady:].mean().item())
    num_knetR_ss.append(frob_errors_knetR[i][idx_steady:].mean().item())
    num_mdm_ss.append(frob_errors_mdm[i][idx_steady:].mean().item())
    num_mehra_ss.append(frob_errors_mehra[i][idx_steady:].mean().item())

axes[-1].set_xlabel("Časový krok [k]")
plt.tight_layout()
plt.show()

# --- Výpis numerických výsledků ---
print("\n" + "="*85)
```

```
print("NUMERICKÉ POROVNÁNÍ KALMANOVA ZISKU V USTÁLENÉM STAVU (Model Mismatch)")
print(f"Hodnoceno jako průměrná Frobeniova norma z posledních␣
↪{steady_state_ratio*100:.0f} % kroků.")
print("CÍL: Číslo co nejblíže k nule (AI by mělo překonat chybný statický KF)")
print("="*85)
print(f"{'Filtr':<35} | {'Průměrná SS Chyba (norma)':<25}")
print("-" * 85)
print(f"{'Statický KF (Nepřesný model)':<35} | {np.mean(num_kf_m_ss):<25.6f}␣
↪(Baseline)")
print(f"{'AKF (MDM)':<35} | {np.mean(num_mdm_ss):<25.6f}")
print(f"{'AKF (Mehra)':<35} | {np.mean(num_mehra_ss):<25.6f}")
print(f"{'Bayesian KNet (BKN)':<35} | {np.mean(num_bkn_ss):<25.6f}")
print(f"{'KNet with Known R':<35} | {np.mean(num_knetR_ss):<25.6f}")
print("="*85)
```

Předpočítávám výsledky filtrů a extrahuji Kalmanovy zisky pro vykreslení…

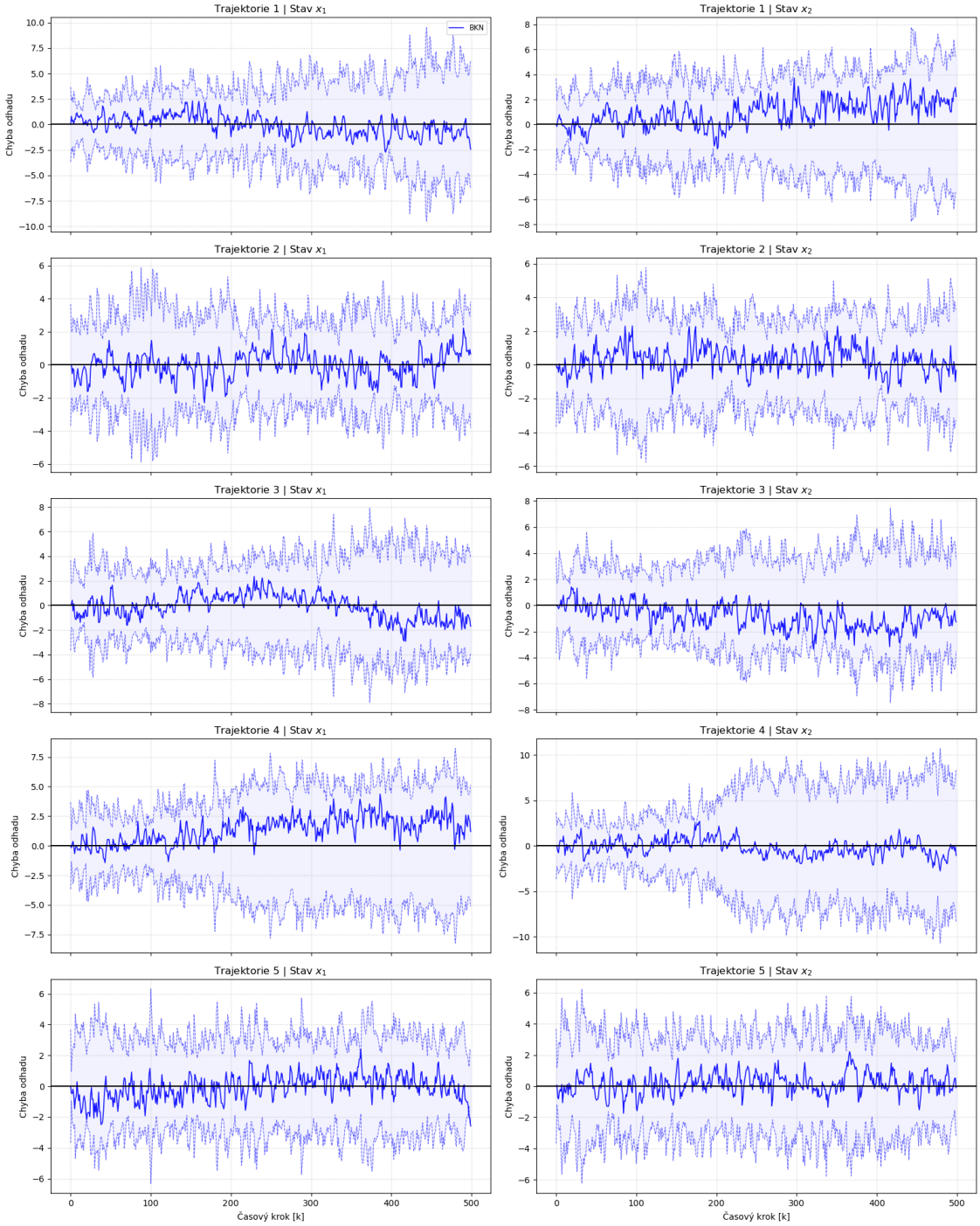
```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[10], line 43
     40 K_kf_m = res_kf_m['Kalman_gain'].cpu()
     42 # --- 3. AKF MDM (Nepřesný model) ---
---> 43 res_mdm,_,_ = akf_mdm_mismatched.
 ↪process_sequence_adaptively(y_test_seq_gpu)
     44 K_mdm = res_mdm['Kalman_gain'].cpu()
     46 # --- 4. AKF Mehra (Nepřesný model) ---

NameError: name 'akf_mdm_mismatched' is not defined
```

[11]:
```
import matplotlib.pyplot as plt

# ==============================================================================
# 6. VYKRESLENÍ VÝVOJE CHYBY A 3-SIGMA OBÁLEK
# ==============================================================================
num_plot_traj = min(5, NUM_TEST_TRAJ)
state_dim = all_x_true_cpu[0].shape[1]
time_steps = np.arange(TEST_SEQ_LEN)

print(f"\nGeneruji grafy vývoje chyby a 3-sigma obálek pro prvních␣
↪{num_plot_traj} trajektorií...")

# Barevná paleta pro jednotlivé modely
colors = {
    'BKN': 'blue',
    'KNetR': 'cyan',
```

```
    'EKF': 'red',
    'UKF': 'green',
    'PF': 'purple'
}

fig, axes = plt.subplots(num_plot_traj, state_dim, figsize=(16, 4 *␣
 ↪num_plot_traj), sharex=True)

# Ošetření pro případ, že vykreslujeme jen 1 trajektorii
if num_plot_traj == 1:
    axes = np.expand_dims(axes, axis=0)

for i in range(num_plot_traj):
    x_true = all_x_true_cpu[i].numpy()

    # Slovník se všemi výsledky pro aktuální trajektorii
    models_data = {
        'BKN': (all_x_hat_bkn_cpu[i].numpy(), all_P_hat_bkn_cpu[i].numpy()),
        # 'KNetR': (all_x_hat_knetR_cpu[i].numpy(), all_P_hat_knetR_cpu[i].
 ↪numpy()),
        # 'EKF': (all_x_hat_ekf_ideal_cpu[i].numpy(),␣
 ↪all_P_hat_ekf_ideal_cpu[i].numpy()),
        # 'UKF': (all_x_hat_ukf_ideal_cpu[i].numpy(),␣
 ↪all_P_hat_ukf_ideal_cpu[i].numpy()),
        # 'PF': (all_x_hat_pf_ideal_cpu[i].numpy(), all_P_hat_pf_ideal_cpu[i].
 ↪numpy())
    }

    for d in range(state_dim):
        ax = axes[i, d]

        # Referenční nulová čára (Ideální stav bez chyby)
        ax.axhline(0, color='black', linestyle='-', linewidth=1.5, zorder=3)

        for model_name, (x_hat, P_hat) in models_data.items():
            # Výpočet chyby: e_t = x_hat - x_true
            error = x_hat[:, d] - x_true[:, d]

            # Výpočet směrodatné odchylky sigma z diagonály kovarianční matice P
            # Použito np.abs pro numerickou stabilitu (P_hat by mělo být␣
 ↪pozitivně definitní)
            std_dev = np.sqrt(np.abs(P_hat[:, d, d]))

            color = colors[model_name]

            # Vykreslení chybové křivky
```

```
            ax.plot(time_steps, error, label=f'{model_name}', color=color,␣
↪linewidth=1.2, alpha=0.9)

            # Vykreslení spojité 3-sigma obálky (centrováno kolem nuly)
            ax.fill_between(time_steps, -3 * std_dev, 3 * std_dev, color=color,␣
↪alpha=0.05)
            ax.plot(time_steps, 3 * std_dev, color=color, linestyle='--',␣
↪linewidth=0.8, alpha=0.6)
            ax.plot(time_steps, -3 * std_dev, color=color, linestyle='--',␣
↪linewidth=0.8, alpha=0.6)

        ax.set_title(f"Trajektorie {i+1} | Stav $x_{d+1}$")
        ax.set_ylabel("Chyba odhadu")
        ax.grid(True, linestyle=':', alpha=0.7)

        # Omezení osy Y pro lepší čitelnost (odřízne obrovské ustřelení EKF)
        # ax.set_ylim([-5, 5]) # Tuto řádku můžeš odkomentovat, pokud bude EKF␣
↪kazit měřítko grafu

        if i == 0 and d == 0:
            ax.legend(loc='upper right', ncol=2, fontsize='small')

# Nastavení osy X jen pro spodní řádek
for d in range(state_dim):
    axes[-1, d].set_xlabel("Časový krok [k]")

plt.tight_layout()
plt.show()
```

```
Generuji grafy vývoje chyby a 3-sigma obálek pro prvních 5 trajektorií…
```


Trajektorie 1 | Stav x_1
Trajektorie 1 | Stav x_2
Trajektorie 2 | Stav x_1
Trajektorie 2 | Stav x_2
Trajektorie 3 | Stav x_1
Trajektorie 3 | Stav x_2
Trajektorie 4 | Stav x_1
Trajektorie 4 | Stav x_2
Trajektorie 5 | Stav x_1
Trajektorie 5 | Stav x_2
BKN
Chyba odhadu
Časový krok [k]